ChatPulse: Real-Time Chat App Project Report

Date: 2026-05-17

Executive Summary

ChatPulse is a full-stack, real-time chat application that combines traditional messaging with live presence updates and an optional AI assistant. The project delivers a polished, responsive user experience on the frontend and a secure, scalable API on the backend. It supports direct chats, group conversations, and an AI chat channel that can answer user prompts through a hosted LLM provider. The system is designed for quick onboarding, minimal latency, and clean separation of concerns between user interface, application logic, and data persistence.

Product Goals

- Provide a modern, real-time messaging experience with low friction login and onboarding.
- Support both one-to-one and group conversations with reliable message delivery.
- Offer an AI assistant chat as a value-added feature for help and quick answers.
- Maintain clear, maintainable architecture for future expansion and deployment.

Architecture and Tech Stack

- Frontend: React 18 with Vite for fast builds, React Router for navigation, Tailwind CSS for UI styling, Socket.IO client for real-time events.
- Backend: Node.js with Express, Socket.IO server for live events, JWT for authentication, Mongoose for MongoDB data access.
- Database: MongoDB with schemas for users, chats, and messages.
- External AI: Groq API for LLM responses, with a dedicated AI user identity inside the chat system.

High-level flow:

- The frontend authenticates users and stores a JWT locally.
- Authenticated requests go through protected API routes.
- Socket connections are established per user for presence, typing, and message delivery.
- Messages are persisted to MongoDB and broadcast through Socket.IO.

Core Features and Workflows

- Authentication: Register and login with hashed passwords and JWT sessions.
- Direct chat: Start a chat by searching users and opening a private thread.
- Group chat: Create groups, rename them, and manage membership with admin rules.
- Live messaging: Messages stream in real time with typing indicators and unread counts.

- AI assistant: Users can open a dedicated AI chat or use the /ai command in a thread.

Typical user flow:

1) Register or log in.

2) Search for a user or create a group.

3) Exchange messages in real time; the chat list updates with latest activity.

4) Optionally invoke AI responses for help or quick answers.

Implementation Highlights

- Socket rooms are organized by user ID and chat ID to ensure correct targeting.
- The backend normalizes CORS origins for safe cross-origin access.
- The frontend keeps chat ordering current by inserting latest messages at the top.
- The system supports typing indicators for both users and AI responses.

Data Model Summary

- User: name, email, password hash, profile picture, timestamps.
- Chat: users, group admin, group flag, latest message, timestamps.
- Message: sender, content, chat reference, timestamps.

These models allow efficient querying for recent chats, message history, and group management.

Security and Privacy

- Passwords are hashed with bcrypt before storage.
- JWT tokens protect API routes and identify users.
- CORS and allowed origins are configurable through environment variables.
- The AI integration uses an API key stored in the server environment.

Deployment and Configuration

- Backend expects MONGO_URI, JWT_SECRET, and optional GROQ_API_KEY and model name.
- Frontend expects VITE_API_BASE_URL and VITE_SOCKET_URL for environment targeting.
- Default development setup runs the API on port 5000 and the client on 5173.

Risks and Future Work

- Add rate limiting and audit logging for production readiness.
- Add file sharing, read receipts, and message reactions.
- Improve moderation controls for group administrators.

- Add automated tests for API routes and socket event flows.
- Package and deploy via containerization and CI/CD for consistent releases.